SCHOOL OF TECHNOLOGY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Java / Python

PROJECT TITLE

HOSTEL ACCOMMODATION AND STUDENT SERVICES MANAGEMENT SYSTEM

Program	**B.Tech**
Section	AIML LEOPARDS
Faculty	Dr. C. Nagaraju
Date of Submission	14 Oct 2026
Academic Year	2026 – 27

Submitted by

Roll Number	Student Name	Mobile	Email ID
25WU0102199	P. Ramsai Yadav	9392616439	peruguramsai@gmail.com

ACKNOWLEDGEMENT

I express my sincere gratitude to **Woxsen University, School of Technology**, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the **Database Management Systems (DBMS)** course.

I thank my course faculty, Dr. C. Nagaraju, for the guidance, suggestions, and support throughout the design and development of this project. The guidance helped me understand the practical application of database concepts such as ER modelling, relational schema design, normalization, SQL, constraints, queries, and database application development.

This PBL activity has strengthened my understanding of DBMS concepts and provided valuable experience in applying them to a real-world problem.

UNDERTAKING

I hereby declare that the project entitled **"Hostel Accommodation and Student Services Management System"**, submitted as part of the Project-Based Learning activity for the **Database Management Systems (DBMS)** course, is my individual work carried out under the guidance of the course faculty.

I undertake that the project report, database design, SQL scripts, application source code, results, and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets, or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Date	**:**		**Name:**	**P. Ramsai Yadav**
Place	:		Roll Number:	25WU0102199
			Program:	B.Tech
			Signature:	

TABLE OF CONTENTS

Contents	Page No.
Abstract	8
LIST OF TABLES	5
LIST OF FIGURES	6
LIST OF ACRONYMS	7
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	9
1.1 Problem Statement	9
1.2 Objectives	9
1.3 Scope and Assumptions	9
1.4 Proposed Solution Strategy	10
1.5 Functional Requirements	10
1.6 Entity-Relationship Diagram (ERD)	11
1.7 Entity and Relationship Summary	12
2. LOGICAL DATABASE DESIGN	14
2.1 Relational Schema	14
2.2 Keys and Integrity Constraints	14
2.3 Functional Dependencies	16
2.4 Normalization up to Third Normal Form (3NF)	16
2.5 Data Dictionary	17
2.6 Implementation of Logical Design: DDL and Sample Data	19
3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN	21
3.1 Application Overview	21
3.2 Development Environment and Database Connectivity	21
3.3 Application Modules and Database Mapping	21
3.4 CRUD Operations	22
3.5 Business Rules, Validation and Error Handling	22
3.6 Reports and Database Operations	23
3.7 Application Screens	23
3.8 Testing and Results	27
4. CONCLUSION	29
4.1 Limitations and Future Enhancements	29
5. REFERENCES	30
6. APPENDICES	31

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	10
Table 2	Proposed Solution Strategy	10
Table 3	Functional Requirements	10
Table 4	Entity and Relationship Summary	12
Table 5	Relational Schema	14
Table 6	Keys and Integrity Constraints	15
Table 7	Normalization Status up to 3NF	17
Table 8	Data Dictionary	17
Table 9	Database Implementation Environment	19
Table 10	Application Development Environment	21
Table 11	Application Module-to-Database Mapping	21
Table 12	CRUD Operations	22
Table 13	Business Rules, Validation and Error Handling	23
Table 14	Reports and Database Operations	23
Table 15	Test Cases	27

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Entity-Relationship Diagram of the Proposed System	12
Figure 2	Dashboard showing occupancy and vacant beds	24
Figure 3	Student registration form and search results	24
Figure 4	Validation error shown for an invalid phone number	25
Figure 5	Bed allocation, vacating and allocation history	25
Figure 6	Transfer form and transfer log after an atomic transfer	26
Figure 7	Fee collection form and outstanding dues	26
Figure 8	Management reports (occupancy, vacancies, dues)	27

LIST OF ACRONYMS

Acronym	Expansion
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form
BCNF	Boyce-Codd Normal Form
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
ER	Entity-Relationship
ERD	Entity-Relationship Diagram
FD	Functional Dependency
FK	Foreign Key
HTML	HyperText Markup Language
IEEE	Institute of Electrical and Electronics Engineers
PBL	Project-Based Learning
PK	Primary Key
SQL	Structured Query Language
UPI	Unified Payments Interface

ABSTRACT

Hostels in educational institutions still depend on registers and spreadsheets to assign beds, collect fees, record visitors and handle complaints. Such manual handling can place more students in a room than it can hold, assign one bed to two residents and leave fee dues and repair requests untracked. The objective of this project is to design and implement a relational database and a small web application that automate hostel accommodation and student services while enforcing these rules inside the database. The system was modelled with an entity-relationship diagram of twelve entities, converted into a relational schema and normalized to Third Normal Form. It was implemented in SQLite using primary key, foreign key, unique, not null, check and default constraints, triggers, views and indexes, and is accessed through a Python Flask application that uses parameterised queries. The application supports student registration, bed allocation, vacating, transfer, fee collection, visitor entry, complaint tracking, maintenance requests and occupancy, vacancy and fee-due reports. Fifteen positive and negative test cases confirmed that duplicate bed assignment, double allocation, overpayment and invalid input are rejected correctly. The completed system provides a reliable, easy-to-use solution that improves data integrity and administrative efficiency.

KEYWORDS

Keywords: *Hostel Management, Relational Database, ER Diagram, Normalization, SQL, Python Flask*

CHAPTER 1
PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

Hostels at colleges and universities manage blocks, rooms, beds, students, fees, visitors, complaints and repairs. In many institutions these activities are still handled in paper registers and separate spreadsheets. Wardens and hostel office staff must manually check which beds are free, which students already hold a bed and whether a room is full. Because nothing prevents a clerical slip, a bed may be assigned to two students, a room may exceed its capacity, a student may appear in two rooms, and a male student may be placed in a girls' block.

The same manual process makes fee follow-up, visitor control, complaint resolution and maintenance tracking unreliable. Payments can be recorded above the amount due, visitor exit times are lost, and unresolved complaints or pending repairs are not visible to management. Students, wardens, the hostel office and the maintenance team all need a single, consistent source of data. A database-backed application that enforces the hostel's business rules at the data level is therefore required to remove duplication, improve accuracy and provide instant reports.

1.2 Objectives

- Design an ER model and a normalized (3NF) relational schema covering blocks, rooms, beds, students, allocations, transfers, vacating records, fees, payments, visitors, complaints and maintenance requests.
- Implement the schema in SQLite with PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK and DEFAULT constraints, triggers and indexes so that double allocation, over-capacity and overpayment are impossible.
- Develop a Python Flask application that performs registration, bed allocation, transfer, fee collection, visitor entry, complaint tracking and maintenance management through parameterised SQL.
- Generate occupancy, vacancy, fee-due, allocation-history, unresolved-complaint and maintenance-status reports, and validate the system through positive and negative test cases.

1.3 Scope and Assumptions

Table 1 defines the boundaries of the project and the assumptions made during design.

Table 1. Scope and Assumptions

Item	Description
In Scope	Hostel blocks, rooms and beds; student registration; bed allocation, vacating and transfer; fee schedules, payment collection and dues; visitor entry and exit; complaint tracking; maintenance requests; occupancy, vacancy and other reports; input validation and database-level integrity rules.
Out of Scope / Exclusions	Online payment gateway, biometric or RFID access control, mess/menu management, multi-campus deployment, mobile app, and email/SMS notifications.
Assumptions	Each block is reserved for one gender; every room has a fixed capacity and one bed record per place; a student holds at most one active bed; fees are fixed per schedule item; the application is used by hostel office staff on a single-server SQLite database.

1.4 Proposed Solution Strategy

The proposed solution is a relational database in which every hostel fact is stored once and linked through keys, following the relational model [1]. A web application sits on top of this database and presents the daily tasks of hostel staff as simple forms. Table 2 summarises the intended users, modules, workflow, rules and outputs.

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Hostel Administrator (full access), Warden (allocation, transfer, complaints, visitors), Accounts Clerk (fee collection and dues), Maintenance Staff (repair requests), Student (registers complaints and views own status through staff).
Major Application Modules	Dashboard; Student Registration; Bed Allocation and Vacating; Transfer; Fee Collection; Visitor Entry; Complaint Tracking; Maintenance; Reports.
Proposed Workflow	Register student -> allocate a vacant bed -> collect fees against schedules -> record visitors, complaints and repairs during stay -> transfer to another bed if required -> vacate with a vacating record at the end of the stay.
Important Business Rules	One active resident per bed; one active allocation per student; room capacity cannot be exceeded; student gender must match block gender; allocation, transfer and vacating dates must be valid; total payment for a fee cannot exceed its amount; students with history cannot be deleted.
Expected Reports / Outputs	Occupancy by block, vacant beds, rooms with vacancies, fee dues, allocation history, students without a bed, visitors currently inside, unresolved complaints and maintenance status summary.

1.5 Functional Requirements

The functional requirements derived from the problem statement are listed in Table 3.

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	Register, search, update and (when no history exists) delete student records with validated email and phone.	High

Req. ID	Functional Requirement	Remarks / Priority
FR-02	Maintain hostel blocks, rooms and beds with fixed room capacity.	High
FR-03	Allocate a vacant bed to a student with start date; reject duplicate bed or student allocation.	High
FR-04	Vacate a student, record the vacating reason and end the allocation.	High
FR-05	Transfer a resident to another vacant bed as one atomic transaction and log it.	High
FR-06	Define fee schedules and collect payments; block payments above the amount due.	High
FR-07	Display fee dues for every student and fee item.	High
FR-08	Record visitor check-in and check-out against a student.	Medium
FR-09	Register complaints and update their status (OPEN, IN_PROGRESS, RESOLVED).	Medium
FR-10	Raise maintenance requests for rooms and track status to completion.	Medium
FR-11	Generate occupancy, vacancy, allocation history and other reports.	High
FR-12	Validate all inputs and display clear error messages for rule violations.	High

1.6 Entity-Relationship Diagram (ERD)

Figure 1 shows the final ER diagram using Chen-style notation [2]. It contains twelve entities and their relationships with cardinalities. Primary keys (PK) and foreign keys (FK) are marked in each entity and match the implemented relational schema. Participation of ALLOCATION and PAYMENT is total with respect to STUDENT, BED and FEE_SCHEDULE, while VACATING is optional for an allocation (0..1).

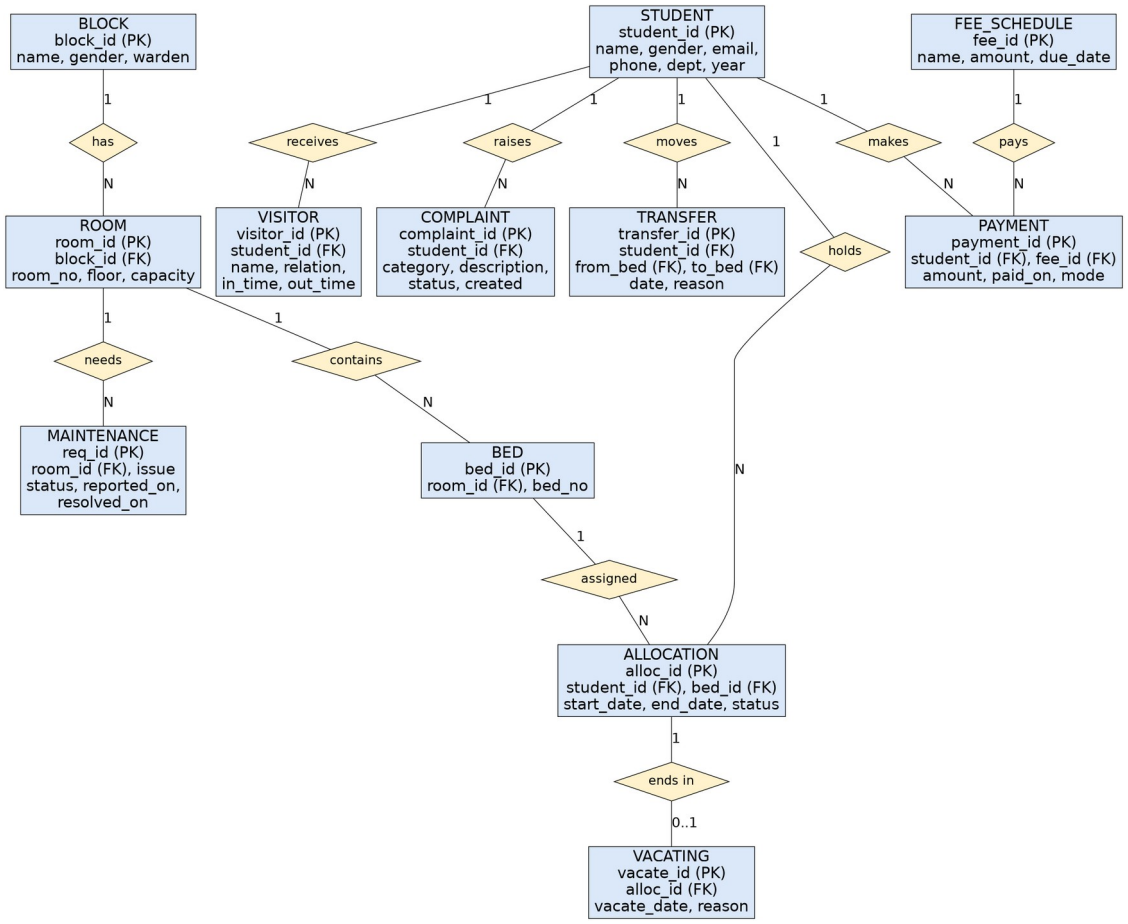


Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

Table 4 summarises each entity, its major attributes and its cardinality with related entities.

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
BLOCK	block_id, name, gender, warden. 1 block : N rooms	Separates boys' and girls' hostels; gender drives allocation rule.
ROOM	room_id, block_id, room_no, floor, capacity. 1 room : N beds, 1 room : N maintenance requests	Physical room with a fixed capacity.
BED	bed_id, room_id, bed_no. 1 bed : N allocations (over time)	Smallest allocatable unit; only one ACTIVE allocation at a time.
STUDENT	student_id, name, gender, email, phone, dept, year. 1 student : N allocations, payments, visitors, complaints	Resident master record.
ALLOCATION	alloc_id, student_id, bed_id, start_date, end_date, status. N : 1 student, N : 1 bed	Resolves the student-bed relationship; keeps full history.
TRANSFER	transfer_id, student_id, from_bed, to_bed, transfer_date, reason	Audit of bed changes.
VACATING	vacate_id, alloc_id (unique), vacate_date, reason. 0..1 per allocation	Records the end of a stay.

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
FEE_SCHEDULE	fee_id, name, amount, due_date. 1 fee : N payments	Defines each chargeable fee item.
PAYMENT	payment_id, student_id, fee_id, amount, paid_on, mode. N : 1 student, N : 1 fee	Resolves the student-fee relationship; total limited by fee amount.
VISITOR	visitor_id, student_id, name, relation, in_time, out_time. N : 1 student	Visitor gate log.
COMPLAINT	complaint_id, student_id, category, description, status, created. N : 1 student	Complaint register with status tracking.
MAINTENANCE	req_id, room_id, issue, status, reported_on, resolved_on. N : 1 room	Repair request tracking.

CHAPTER 2
LOGICAL DATABASE DESIGN

2.1 Relational Schema

The ER model was converted into twelve relations. One-to-many relationships were mapped by placing the primary key of the "one" side as a foreign key on the "many" side. The many-to-many relationships student-bed and student-fee were resolved into the relations allocation and payment. Table 5 lists each relation with its primary key (PK) and foreign keys (FK).

Table 5. Relational Schema

S.No.	Table Name	Purpose
1	block (block_id PK, name, gender, warden)	Hostel blocks; gender restricts allocation.
2	room (room_id PK, block_id FK, room_no, floor, capacity)	Rooms inside a block with capacity 1-6.
3	bed (bed_id PK, room_id FK, bed_no)	Individual beds of a room.
4	student (student_id PK, name, gender, email, phone, dept, year, joined)	Student master data.
5	allocation (alloc_id PK, student_id FK, bed_id FK, start_date, end_date, status)	Bed assignments with history.
6	transfer (transfer_id PK, student_id FK, from_bed FK, to_bed FK, transfer_date, reason)	Bed transfer log.
7	vacating (vacate_id PK, alloc_id FK, vacate_date, reason)	Vacating records.
8	fee_schedule (fee_id PK, name, amount, due_date)	Fee items and amounts.
9	payment (payment_id PK, student_id FK, fee_id FK, amount, paid_on, mode)	Fee payments received.
10	visitor (visitor_id PK, student_id FK, name, relation, in_time, out_time)	Visitor entries.
11	complaint (complaint_id PK, student_id FK, category, description, status, created)	Student complaints.
12	maintenance (req_id PK, room_id FK, issue, status, reported_on, resolved_on)	Room repair requests.
13	v_vacant_beds, v_occupancy, v_fee_dues (views)	Derived views for vacancies, occupancy and dues.

2.2 Keys and Integrity Constraints

Table 6 lists the keys and the business rules enforced through constraints. Referential integrity uses the default RESTRICT behaviour (PRAGMA foreign_keys = ON), so a parent row cannot be deleted while child rows exist.

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
block	Primary Key	block_id	Unique block identity
	Unique / Check	name; gender IN (M,F)	No duplicate block; valid gender
room	Primary Key	room_id	
	Foreign Key	block_id -> block	Room belongs to a block
	Unique / Check	(block_id, room_no); capacity BETWEEN 1 AND 6; floor >= 0	No duplicate room; capacity control
bed	Primary Key	bed_id	
	Foreign Key	room_id -> room	
	Unique	(room_id, bed_no)	No duplicate bed number
	Trigger	trg_bed_capacity	Beds in a room cannot exceed capacity
student	Primary Key	student_id	
	Unique	email	One account per email
	Check	gender IN (M,F); length(phone)=10; year BETWEEN 1 AND 5	Valid data
	Default	joined = CURRENT_DATE	
allocation	Primary Key	alloc_id	
	Foreign Key	student_id -> student; bed_id -> bed	
	Partial Unique	bed_id WHERE status=ACTIVE	One active resident per bed
	Partial Unique	student_id WHERE status=ACTIVE	One active allocation per student
	Check	end_date IS NULL OR end_date >= start_date; status IN (ACTIVE,ENDED)	Valid allocation/vacating dates
	Trigger	trg_alloc_gender	Student gender must match block
	Default	start_date = CURRENT_DATE; status = ACTIVE	
transfer	Foreign Key	student_id, from_bed, to_bed	Transfer references real beds
	Check	from_bed <> to_bed	Real change of bed
vacating	Foreign Key / Unique	alloc_id -> allocation (UNIQUE)	One vacating record per allocation
fee_schedule	Unique / Check	name; amount > 0	No duplicate fee; positive amount
payment	Foreign Key	student_id -> student; fee_id -> fee_schedule	
	Check / Default	amount > 0; mode IN (CASH,UPI,CARD,BANK); paid_on = CURRENT_DATE	Valid payment
	Trigger	trg_pay_limit	Total paid cannot exceed fee amount
visitor	Foreign Key / Check	student_id -> student; out_time >= in_time	Valid visit duration

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
complaint	Foreign Key / Check	student_id -> student; status IN (OPEN,IN_PROGRESS,RESOLVED)	Controlled status values
maintenance	Foreign Key / Check	room_id -> room; status IN (PENDING,IN_PROGRESS,DONE); resolved_on >= reported_on	Valid repair tracking

2.3 Functional Dependencies

The important functional dependencies used to justify normalization are given below; the left side of every dependency is a candidate key of its relation.

- `block: block_id -> name, gender, warden; name -> block_id`
- `room: room_id -> block_id, room_no, floor, capacity; (block_id, room_no) -> room_id`
- `bed: bed_id -> room_id, bed_no; (room_id, bed_no) -> bed_id`
- `student: student_id -> name, gender, email, phone, dept, year, joined; email -> student_id`
- `allocation: alloc_id -> student_id, bed_id, start_date, end_date, status`
- `transfer: transfer_id -> student_id, from_bed, to_bed, transfer_date, reason`
- `vacating: vacate_id -> alloc_id, vacate_date, reason; alloc_id -> vacate_id`
- `fee_schedule: fee_id -> name, amount, due_date; name -> fee_id`
- `payment: payment_id -> student_id, fee_id, amount, paid_on, mode`
- `visitor: visitor_id -> student_id, name, relation, in_time, out_time`
- `complaint: complaint_id -> student_id, category, description, status, created`
- `maintenance: req_id -> room_id, issue, status, reported_on, resolved_on`

2.4 Normalization up to Third Normal Form (3NF)

Normalization follows the standard definitions [3], [4]. The manual hostel register can be thought of as one wide table: REGISTER(student, email, phone, bed, room, block, warden, block gender, fee, fee amount, amount paid, complaint, visitor). It repeats student, room and warden details in every row and causes update, insertion and deletion anomalies. Repeating groups (many payments, visitors and complaints per student) violate 1NF and were moved to separate relations. Partial dependencies (room details depending only on part of the key) were removed for 2NF by creating room, bed and fee_schedule. Transitive dependencies such as bed -> room -> block -> warden were removed for 3NF by separating block, room and bed. Table 7 shows the final status.

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
block	Yes	Yes	Yes	Yes	Only candidate keys (block_id, name) determine attributes.
room	Yes	Yes	Yes	Yes	Warden stays in block, not repeated per room.
bed	Yes	Yes	Yes	Yes	Separated from room to allow one row per bed.
student	Yes	Yes	Yes	Yes	Personal data stored once.
allocation	Yes	Yes	Yes	Yes	Resolves student-bed M:N; status kept as controlled redundancy for partial indexes.
transfer	Yes	Yes	Yes	Yes	Audit table; beds referenced by key.
vacating	Yes	Yes	Yes	Yes	Split from allocation to keep reason/date of leaving separate.
fee_schedule	Yes	Yes	Yes	Yes	Fee amount stored once, not repeated in payments.
payment	Yes	Yes	Yes	Yes	Resolves student-fee M:N.
visitor	Yes	Yes	Yes	Yes	Single-valued visit attributes.
complaint	Yes	Yes	Yes	Yes	No transitive dependency.
maintenance	Yes	Yes	Yes	Yes	Room details referenced through room_id.

The column allocation.status is derivable from end_date and is kept as a deliberate, controlled redundancy: it allows the partial unique indexes that guarantee one active resident per bed. A CHECK constraint keeps status and end_date consistent.

2.5 Data Dictionary

Table 8 describes every attribute of the implemented tables.

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
block	block_id	INTEGER	PK	Block identifier
block	name	TEXT	UNIQUE, NOT NULL	Block name
block	gender	TEXT	CHECK (M/F)	Gender housed
block	warden	TEXT	NOT NULL	Warden name
room	room_id	INTEGER	PK	Room identifier
room	block_id	INTEGER	FK, NOT NULL	Owning block
room	room_no	TEXT	NOT NULL	Room number
room	floor	INTEGER	CHECK >= 0	Floor number
room	capacity	INTEGER	CHECK 1-6	Maximum occupants
bed	bed_id	INTEGER	PK	Bed identifier
bed	room_id	INTEGER	FK, NOT NULL	Room of the bed
bed	bed_no	INTEGER	NOT NULL	Bed number in room
student	student_id	INTEGER	PK	Student identifier
student	name	TEXT	NOT NULL	Full name
student	gender	TEXT	CHECK (M/F)	Gender
student	email	TEXT	UNIQUE, NOT NULL	Email address

Table	Attribute	Data Type	Key / Constraint	Description
student	phone	TEXT	CHECK length 10	Mobile number
student	dept	TEXT	NOT NULL	Department
student	year	INTEGER	CHECK 1-5	Year of study
student	joined	DATE	DEFAULT today	Registration date
allocation	alloc_id	INTEGER	PK	Allocation identifier
allocation	student_id	INTEGER	FK, NOT NULL	Resident
allocation	bed_id	INTEGER	FK, NOT NULL	Allocated bed
allocation	start_date	DATE	DEFAULT today	Start of stay
allocation	end_date	DATE	CHECK >= start	End of stay
allocation	status	TEXT	CHECK, DEFAULT ACTIVE	ACTIVE or ENDED
transfer	transfer_id	INTEGER	PK	Transfer identifier
transfer	student_id	INTEGER	FK	Student moved
transfer	from_bed	INTEGER	FK	Previous bed
transfer	to_bed	INTEGER	FK	New bed
transfer	transfer_date	DATE	DEFAULT today	Date of transfer
transfer	reason	TEXT	NOT NULL	Reason
vacating	vacate_id	INTEGER	PK	Vacating identifier
vacating	alloc_id	INTEGER	FK, UNIQUE	Allocation ended
vacating	vacate_date	DATE	DEFAULT today	Date vacated
vacating	reason	TEXT	NOT NULL	Reason
fee_schedule	fee_id	INTEGER	PK	Fee identifier
fee_schedule	name	TEXT	UNIQUE, NOT NULL	Fee name
fee_schedule	amount	REAL	CHECK > 0	Fee amount
fee_schedule	due_date	DATE	NOT NULL	Due date
payment	payment_id	INTEGER	PK	Payment identifier
payment	student_id	INTEGER	FK	Paying student
payment	fee_id	INTEGER	FK	Fee paid
payment	amount	REAL	CHECK > 0	Amount paid
payment	paid_on	DATE	DEFAULT today	Payment date
payment	mode	TEXT	CHECK, DEFAULT CASH	CASH/UPI/CARD/BANK
visitor	visitor_id	INTEGER	PK	Visitor identifier
visitor	student_id	INTEGER	FK	Student visited
visitor	name	TEXT	NOT NULL	Visitor name
visitor	relation	TEXT	NOT NULL	Relation to student
visitor	in_time	TEXT	NOT NULL	Entry time
visitor	out_time	TEXT	CHECK >= in_time	Exit time
complaint	complaint_id	INTEGER	PK	Complaint identifier

Table	Attribute	Data Type	Key / Constraint	Description
complaint	student_id	INTEGER	FK	Complainant
complaint	category	TEXT	NOT NULL	Complaint type
complaint	description	TEXT	NOT NULL	Details
complaint	status	TEXT	CHECK, DEFAULT OPEN	Progress status
complaint	created	DATE	DEFAULT today	Date raised
maintenance	req_id	INTEGER	PK	Request identifier
maintenance	room_id	INTEGER	FK	Room concerned
maintenance	issue	TEXT	NOT NULL	Issue description
maintenance	status	TEXT	CHECK, DEFAULT PENDING	Repair status
maintenance	reported_on	DATE	DEFAULT today	Reported date
maintenance	resolved_on	DATE	CHECK >= reported	Completion date

2.6 Implementation of Logical Design: DDL and Sample Data

The design was implemented in SQLite as described in Table 9. Representative DDL for the allocation and payment tables, including the partial unique indexes and the payment-limit trigger, is shown below; the complete script is in Appendix A.

Table 9. Database Implementation Environment

Item	Details
DBMS	SQLite
DBMS Version	3.45.1 (as tested)
Number of Tables	12 tables, 3 views, 3 triggers, 7 indexes
Sample Data	2 blocks, 8 rooms, 24 beds, 10 students, 8 allocations, 3 fee items, 6 payments, 2 visitors, 3 complaints, 2 maintenance requests

```
CREATE TABLE allocation(
  alloc_id   INTEGER PRIMARY KEY,
  student_id INTEGER NOT NULL REFERENCES student,
  bed_id     INTEGER NOT NULL REFERENCES bed,
  start_date DATE NOT NULL DEFAULT CURRENT_DATE,
  end_date   DATE,
  status     TEXT NOT NULL DEFAULT 'ACTIVE'
             CHECK(status IN('ACTIVE','ENDED')),
  CHECK(end_date IS NULL OR end_date >= start_date));
CREATE UNIQUE INDEX ux_one_active_bed
  ON allocation(bed_id) WHERE status='ACTIVE';
CREATE UNIQUE INDEX ux_one_active_student
  ON allocation(student_id) WHERE status='ACTIVE';

CREATE TABLE payment(
  payment_id INTEGER PRIMARY KEY,
  student_id INTEGER NOT NULL REFERENCES student,
  fee_id     INTEGER NOT NULL REFERENCES fee_schedule,
```

```
  amount     REAL NOT NULL CHECK(amount>0),
  paid_on    DATE NOT NULL DEFAULT CURRENT_DATE,
  mode TEXT NOT NULL DEFAULT 'CASH'
       CHECK(mode IN('CASH','UPI','CARD','BANK')));
CREATE TRIGGER trg_pay_limit BEFORE INSERT ON payment BEGIN
  SELECT RAISE(ABORT,'Payment exceeds the fee amount due')
  WHERE (SELECT COALESCE(SUM(amount),0) FROM payment
         WHERE student_id=NEW.student_id AND fee_id=NEW.fee_id)
        + NEW.amount >
        (SELECT amount FROM fee_schedule WHERE fee_id=NEW.fee_id);
END;
```

Representative sample data inserted by the script (students and their current beds):

```
student_id | name         | gender | dept | year | current bed
1          | Arjun Reddy  | M      | CSE  | 2    | Block A / 101 / Bed 1
2          | Rahul Verma  | M      | ECE  | 1    | Block A / 101 / Bed 2
5          | Ananya Iyer  | F      | IT   | 2    | Block B / 101 / Bed 1
9          | Vikram Patel | M      | CIVIL| 1    | (not allocated)
```

CHAPTER 3
APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The Hostel Accommodation and Student Services Management System is a web application used by the hostel administrator, wardens, accounts clerk and maintenance staff. It replaces paper registers with a single SQLite database accessed through a Python Flask application. Every screen reads from and writes to the database using parameterised SQL; no data is stored statically in the program. The application provides nine screens covering the dashboard, student registration, allocation and vacating, transfer, fee collection, visitor entry, complaints, maintenance and reports. Business rules are enforced twice: first by input validation in Python and then by constraints and triggers inside the database, so that invalid data is rejected even if a screen check is bypassed.

3.2 Development Environment and Database Connectivity

The application is written in Python using the Flask framework [6]. It connects to the SQLite database [5] through the built-in sqlite3 module [7]. A connection is opened for each request with foreign-key enforcement turned on, and is closed when the request ends. Table 10 lists the environment.

Table 10. Application Development Environment

Item	Details
Programming Language	Python 3.12
DBMS	SQLite 3.45.1
IDE / Framework / GUI Toolkit	Flask 3.1 web framework, Jinja2 templates, HTML/CSS front end; VS Code
Database Connector / Driver	Python built-in sqlite3 module (DB-API 2.0)
Operating Environment	Windows / Linux, any modern web browser
Repository Link	[Add GitHub repository link here]

3.3 Application Modules and Database Mapping

Each application screen maps to specific tables and SQL operations, as shown in Table 11.

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Dashboard	Occupancy and vacancy summary	block, room, bed, allocation, student, complaint, maintenance	SELECT on views v_occupancy, v_vacant_beds
Student Registration	Register, search, update, delete students	student	INSERT, SELECT (LIKE), UPDATE, DELETE

Module / Screen	Purpose	Tables Used	Main Operations
Bed Allocation	Allocate and vacate beds, show history	allocation, vacating, bed, student	INSERT, UPDATE, SELECT
Transfer	Move resident to another bed	allocation, transfer	UPDATE + INSERT in one transaction
Fee Collection	Collect payment, add fee, show dues	fee_schedule, payment, student	INSERT, SELECT on v_fee_dues
Visitor Entry	Check-in and check-out visitors	visitor, student	INSERT, UPDATE
Complaint Tracking	Raise and update complaints	complaint, student	INSERT, UPDATE, SELECT
Maintenance	Raise and track repairs	maintenance, room, block	INSERT, UPDATE, SELECT
Reports	Management reports	All tables and views	Joins, GROUP BY, HAVING, nested queries

3.4 CRUD Operations

Table 12 shows how create, read, update and controlled removal are performed by the application.

Table 12. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Create	Register student; allocate bed; collect payment; visitor check-in; raise complaint/maintenance	student, allocation, payment, visitor, complaint, maintenance	Figures 3 and 5; TC-01, TC-04, TC-10
Read / Search	Student search by name, department or email; dashboard; allocation history; dues; reports	student, allocation, v_fee_dues and views	Figures 2, 3, 5, 7 and 8
Update	Update phone; vacate; transfer; visitor check-out; complaint and maintenance status	student, allocation, visitor, complaint, maintenance	Figure 6; TC-09, TC-13, TC-14
Delete / Controlled Removal	Delete student only when no dependent records exist; beds are never deleted, allocations are ended not removed	student	TC-15

3.5 Business Rules, Validation and Error Handling

Input is validated in Python before it reaches the database, and the database independently rejects rule violations. Exceptions raised by SQLite (IntegrityError) and validation errors (ValueError) are caught, the transaction is rolled back and the message is shown on the screen. All SQL statements use parameter placeholders, which prevents SQL injection. Table 13 lists the rules and their implementation.

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
Valid email and 10-digit phone	Regular-expression checks in Python before INSERT/UPDATE; CHECK constraint on phone	"Phone must be exactly 10 digits" (Figure 4)
One active resident per bed / one active allocation per student	Partial unique indexes on allocation WHERE status=ACTIVE	"UNIQUE constraint failed: allocation.bed_id / student_id"
Gender and capacity control	trg_alloc_gender and trg_bed_capacity triggers	"Student gender does not match block gender"
Valid dates	CHECK (end_date >= start_date); date compared with start in Python	"Vacating date cannot be before allocation start date"
Payment limit	trg_pay_limit trigger; amount > 0 CHECK	"Payment exceeds the fee amount due"
Transaction safety and SQL injection	Every request runs in one transaction with commit or rollback; all SQL uses ? placeholders; delete is limited by foreign keys	Failed transfer leaves old allocation unchanged

3.6 Reports and Database Operations

The reports use joins, aggregate functions, GROUP BY with HAVING, nested queries and views, as summarised in Table 14. The SQL statements are given in Appendix A.

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Occupancy by block	Join of block, room, bed, allocation with COUNT and GROUP BY (view v_occupancy)	Table of total, occupied, vacant beds and %
Vacant beds	View v_vacant_beds using NOT EXISTS subquery	List of free beds by block and room
Rooms with vacancies	GROUP BY room with HAVING vacant > 0	Rooms that still have free places
Fee dues	CROSS JOIN fee_schedule with LEFT JOIN payment, SUM aggregate (view v_fee_dues)	Amount, paid and due per student and fee
Students without a bed	Nested query: student_id NOT IN (SELECT ... active allocations)	Unallocated students
Unresolved complaints / maintenance status	WHERE status <> RESOLVED; GROUP BY status	Open complaints and request counts
Allocation history and visitors inside	Multi-table JOIN; WHERE out_time IS NULL	Full history and current visitors

3.7 Application Screens

Figures 2 to 8 show the application working on the database. Figure 3 demonstrates data entry and search, Figure 4 shows validation, Figures 5 and 6 show allocation and an atomic transfer transaction, and Figures 7 and 8 show fee collection and report output.

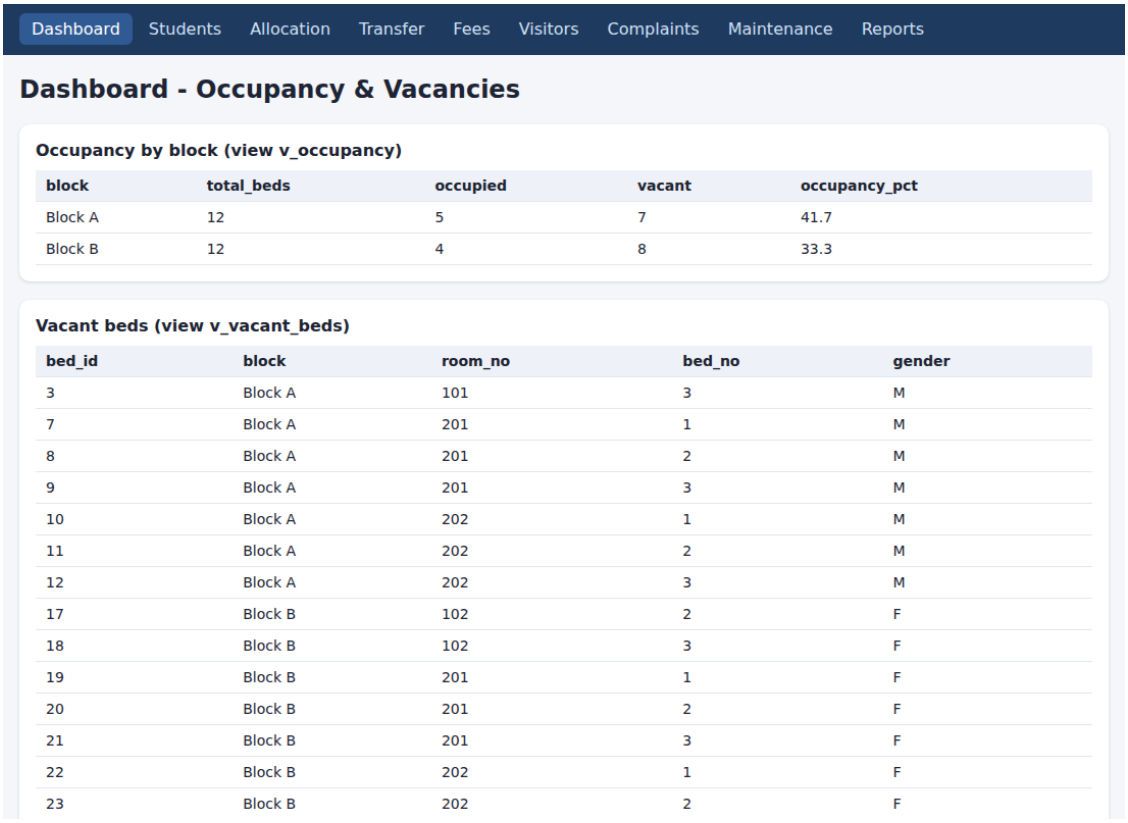


Figure 2. Dashboard showing occupancy and vacant beds

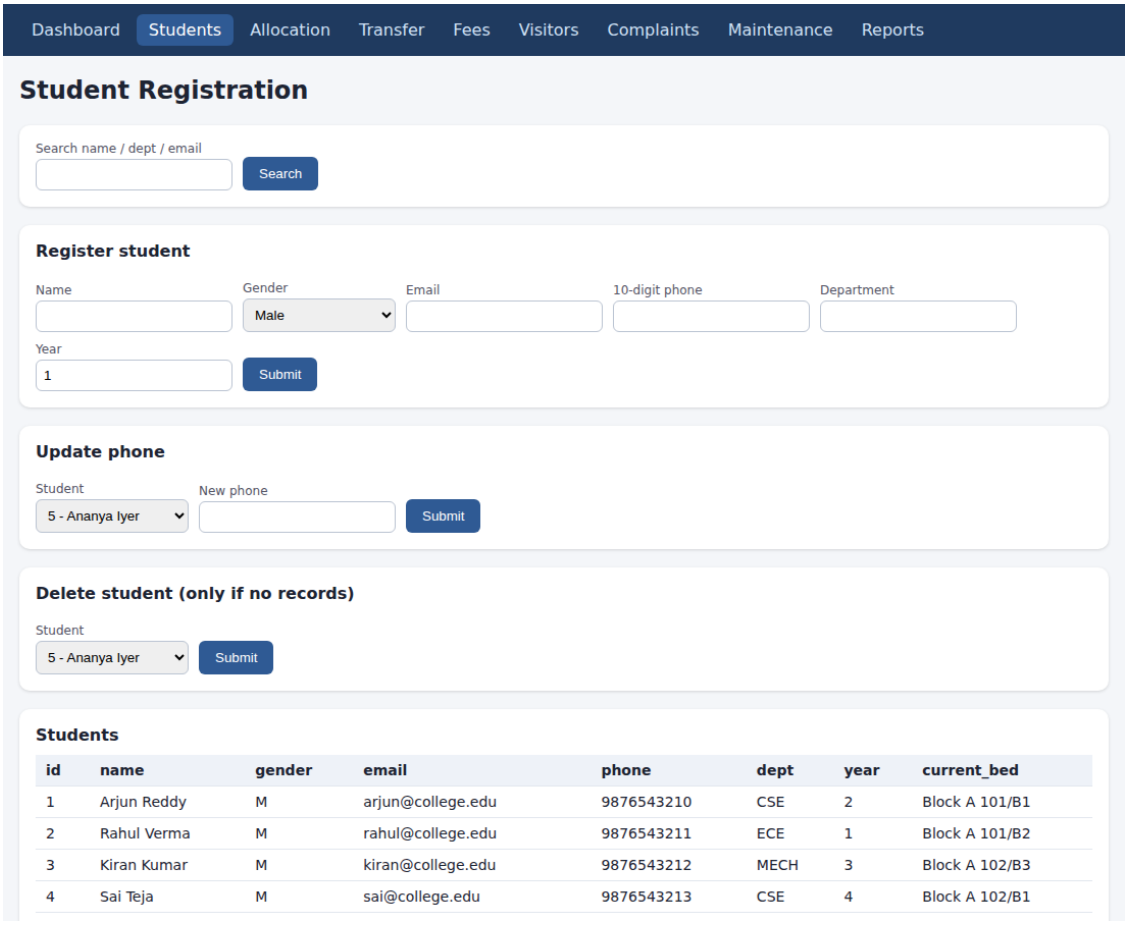


Figure 3. Student registration form and search results

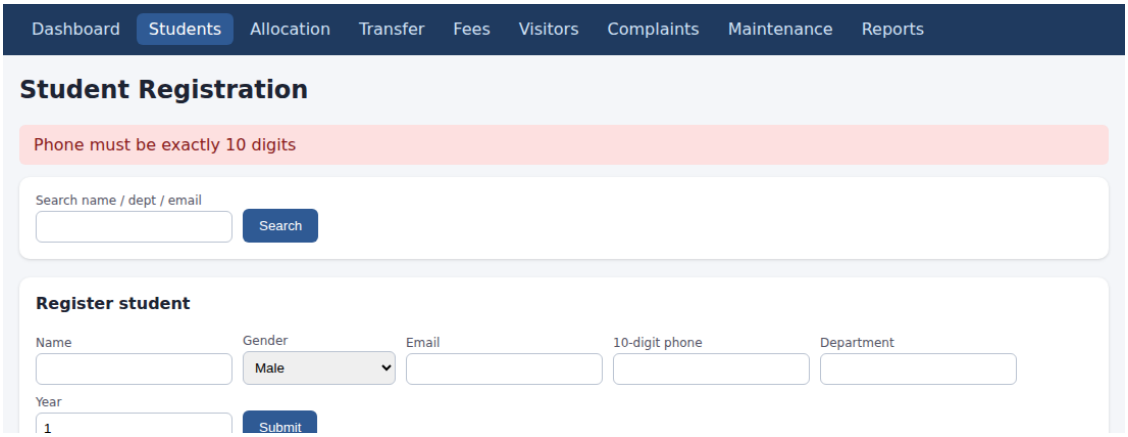


Figure 4. Validation error shown for an invalid phone number

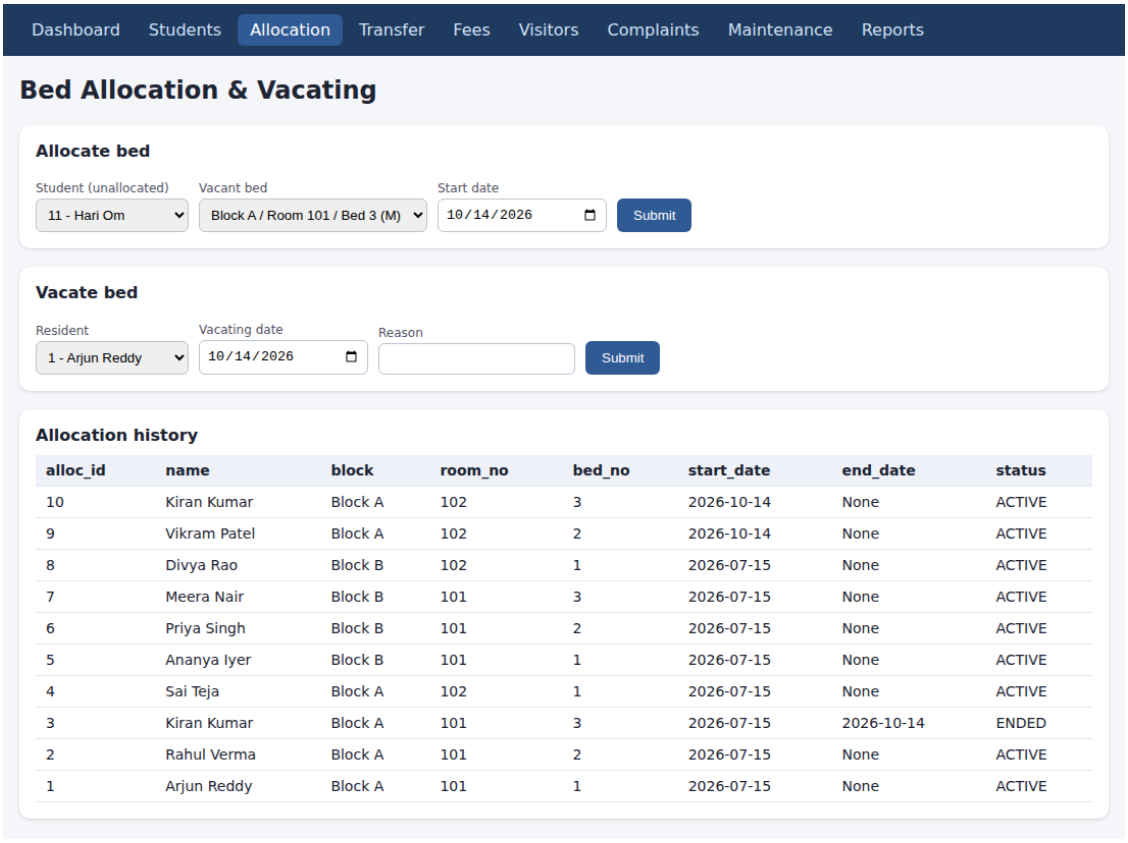


Figure 5. Bed allocation, vacating and allocation history

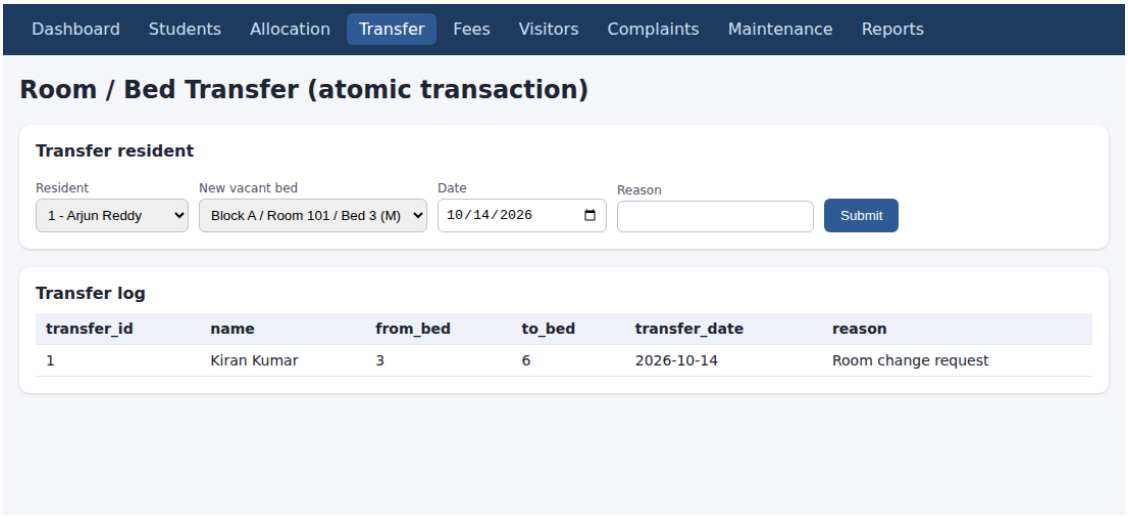


Figure 6. Transfer form and transfer log after an atomic transfer

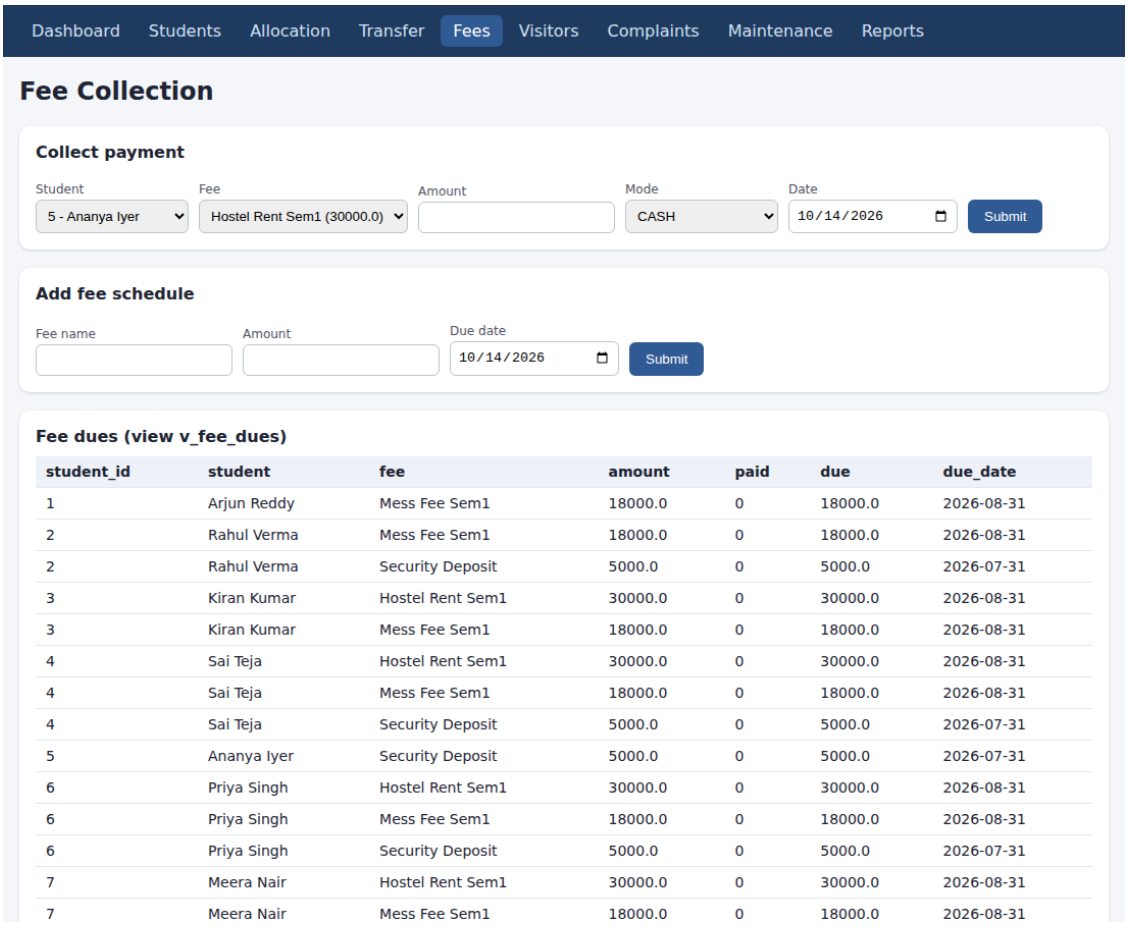


Figure 7. Fee collection form and outstanding dues

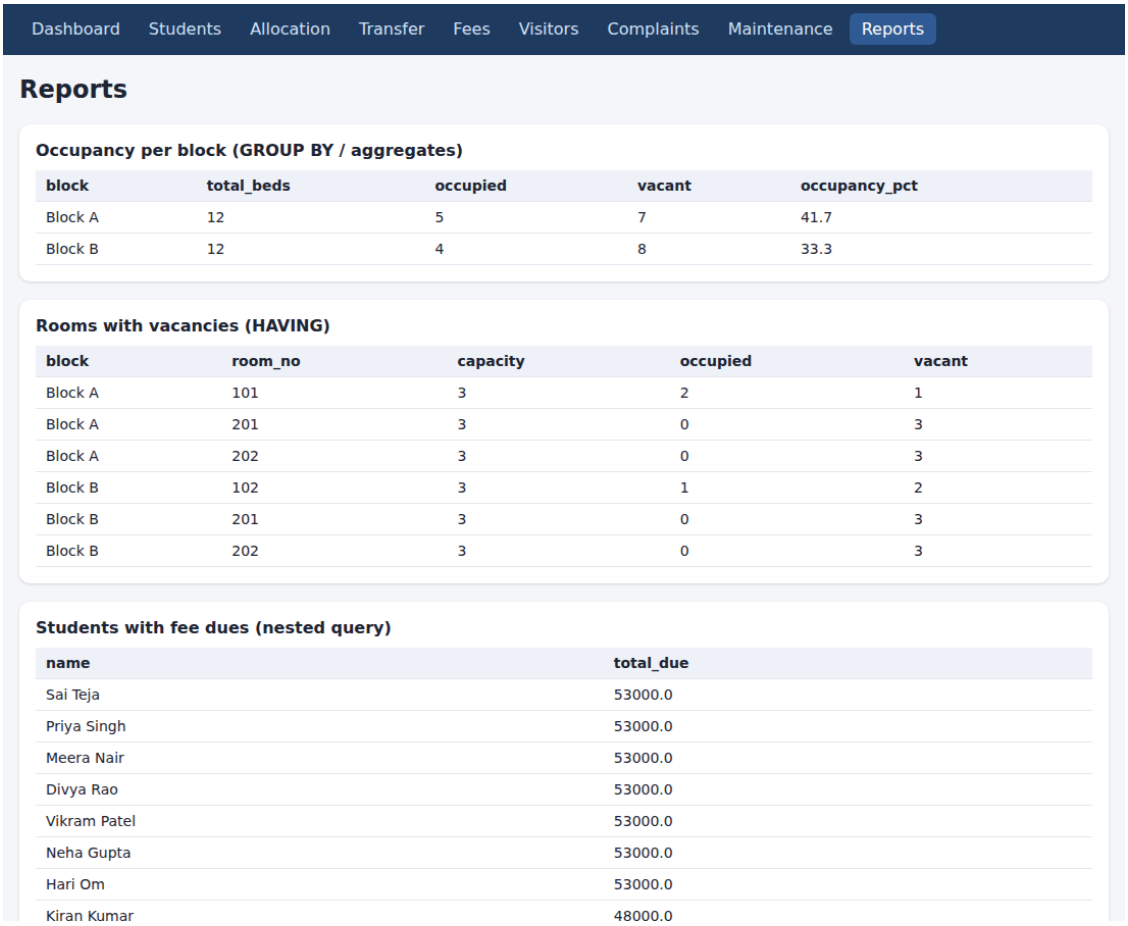


Figure 8. Management reports (occupancy, vacancies, dues)

3.8 Testing and Results

Fifteen positive and negative test cases were executed against the running application and database (script test_cases.py). Table 15 gives the inputs, expected results and the observed outcome.

Table 15. Test Cases

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Student registration	Valid student Hari Om, M, hari@college.edu, 9876543299	Record inserted; success message	Student added; success message	Pass
TC-02	Student registration	Phone "12345" (invalid)	Rejected with phone validation message	Phone must be exactly 10 digits	Pass
TC-03	Student registration	Duplicate email arjun@college.edu	Rejected by UNIQUE constraint on email	UNIQUE constraint failed: student.email	Pass
TC-04	Bed allocation	Allocate student 9 (M) to a vacant bed in Block A	Allocation created and bed becomes occupied	Allocation created; success message	Pass
TC-05	Double allocation	Allocate student 1 (already resident) to another bed	Rejected: one active allocation per student	UNIQUE constraint failed: allocation.student_id	Pass
TC-06	One resident per bed	Allocate student 10 (F) to bed already occupied by student 5	Rejected: one active resident per bed	UNIQUE constraint failed: allocation.bed_id	Pass

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-07	Gender rule	Allocate female student 10 to a Block A bed	Rejected by gender trigger	Student gender does not match block gender	Pass
TC-08	Date validation	Vacate student 2 with date before start (2026-01-01)	Rejected: date before allocation start	Vacating date cannot be before allocation start date	Pass
TC-09	Bed transfer	Transfer student 3 to a vacant Block A bed	Old allocation ended, new one active, transfer logged	Old allocation ENDED (2026-10-14); new ACTIVE allocation; 1 transfer row	Pass
TC-10	Fee payment	Student 2 pays Rs 15000 against Hostel Rent (remaining 15000)	Payment accepted; due becomes zero	Accepted; due = 0.0 in v_fee_dues	Pass
TC-11	Payment limit	Student 2 pays Rs 500 more after fee fully paid	Rejected by payment-limit trigger	Payment exceeds the fee amount due	Pass
TC-12	Visitor entry	Check-in visitor for student 3	Visitor saved with no exit time	Saved; out_time is NULL	Pass
TC-13	Complaint tracking	Update complaint 1 to RESOLVED	Status changed to RESOLVED	Complaint 1 status = RESOLVED	Pass
TC-14	Maintenance	Mark request 1 as DONE	Status DONE and resolved date set	Request 1 status = DONE, resolved 2026-10-14	Pass
TC-15	Controlled delete	Delete student 1 (has allocation/payment records)	Rejected by foreign key constraint	FOREIGN KEY constraint failed	Pass

All 15 test cases passed. Positive cases (registration, allocation, transfer, payment, visitor, complaint and maintenance updates) changed the database as expected, and negative cases were rejected with specific messages from either the application or the database constraints and triggers. This confirms that the application correctly uses the designed database and enforces one active resident per bed, one active allocation per student, gender control, valid dates, the payment limit and referential integrity.

CHAPTER 4
CONCLUSION

This project addressed the problem of manual hostel administration, where duplicate bed assignments, over-capacity rooms, untracked fee dues and unresolved complaints were common. An entity-relationship model of twelve entities was prepared, converted into a relational schema and normalized to Third Normal Form. The schema was implemented in SQLite with primary keys, foreign keys, unique, not null, check and default constraints, three triggers, three views and seven indexes.

A Python Flask application was developed on top of this database to perform student registration, bed allocation, vacating, transfer, fee collection, visitor entry, complaint tracking, maintenance management and reporting. All fifteen test cases produced the expected outcome, showing that the application uses the designed database and that the important business rules are enforced at the database level. The project gave practical experience in ER modelling, normalization, SQL, integrity constraints, transactions and database-driven application development.

4.1 Limitations and Future Enhancements

- The application has no login or role-based access control; user roles are described but not enforced.
- SQLite supports a single-server, low-concurrency setup; a production hostel would use MySQL or PostgreSQL.
- Future work: student self-service portal, online payment gateway, email/SMS reminders for dues, QR-based visitor passes and exportable PDF reports.

5. REFERENCES

[1] E. F. Codd, "A relational model of data for large shared data banks," Commun. ACM, vol. 13, no. 6, pp. 377-387, Jun. 1970, doi: 10.1145/362384.362685.

[2] P. P. Chen, "The entity-relationship model-toward a unified view of data," ACM Trans. Database Syst., vol. 1, no. 1, pp. 9-36, Mar. 1976, doi: 10.1145/320434.320440.

[3] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Hoboken, NJ, USA: Pearson, 2016.

[4] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[5] SQLite Consortium, "SQLite documentation." Accessed: Oct. 14, 2026. [Online]. Available: https://www.sqlite.org/docs.html

[6] Pallets, "Flask documentation," ver. 3.1. Accessed: Oct. 14, 2026. [Online]. Available: https://flask.palletsprojects.com/

[7] Python Software Foundation, "sqlite3 - DB-API 2.0 interface for SQLite databases," Python 3 documentation. Accessed: Oct. 14, 2026. [Online]. Available: https://docs.python.org/3/library/sqlite3.html

6. APPENDICES

APPENDIX A - COMPLETE DATABASE SCRIPT

The complete database script (schema, constraints, triggers, views, indexes and sample data) is submitted as hostel_database.sql. The report queries are listed below.

```
-- Occupancy by block (view)
CREATE VIEW v_occupancy AS
SELECT bl.name block, COUNT(b.bed_id) total_beds,
       COUNT(a.alloc_id) occupied,
       COUNT(b.bed_id)-COUNT(a.alloc_id) vacant
FROM block bl JOIN room r ON r.block_id=bl.block_id
     JOIN bed b ON b.room_id=r.room_id
     LEFT JOIN allocation a ON a.bed_id=b.bed_id AND a.status='ACTIVE'
GROUP BY bl.block_id;

-- Rooms with vacancies
SELECT bl.name, r.room_no, r.capacity - COUNT(a.alloc_id) AS vacant
FROM room r JOIN block bl ON bl.block_id=r.block_id
     JOIN bed b ON b.room_id=r.room_id
     LEFT JOIN allocation a ON a.bed_id=b.bed_id AND a.status='ACTIVE'
GROUP BY r.room_id HAVING vacant > 0;

-- Students with fee dues (nested query)
SELECT name, (SELECT SUM(due) FROM v_fee_dues d
              WHERE d.student_id=s.student_id) AS total_due
FROM student s
WHERE student_id IN (SELECT student_id FROM v_fee_dues WHERE due>0);

-- Students without a bed (nested query)
SELECT student_id, name FROM student
WHERE student_id NOT IN
  (SELECT student_id FROM allocation WHERE status='ACTIVE');

-- Unresolved complaints
SELECT c.complaint_id, s.name, c.category, c.status
FROM complaint c JOIN student s ON s.student_id=c.student_id
WHERE c.status <> 'RESOLVED';
```

APPENDIX B - APPLICATION SOURCE CODE

The complete source code is in the single file hostel_app/app.py (Flask routes, validation and database access). Repository: [Add GitHub repository link here]. Run with: pip install flask, then python app.py, then open http://127.0.0.1:5000.

APPENDIX C - ADDITIONAL TEST DATA AND OUTPUTS

The automated test script hostel_app/test_cases.py executes the 15 cases of Table 15 against a freshly created database using Flask's test client and prints the observed messages. A browser-only demonstration build (hostel.html) with the same rules is also available for presentations without a Python installation.